



Digital IC Design

Python Scripting Assignment

Release: 1.0
Date: 20-May-2025



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	25/04/2025	Musadiq Hussain	Complete manual	-

TABLE OF CONTENTS



1. Objective	3
2. Tools	3
3. Introduction	3
4. Lab Task	4
5. Submission Guidelines	4
6. References	5



1. Objective

- Learn to automate file handling tasks in digital IC design using Python.
- Understand how to manage project directories, generate design templates, and rename, move, or delete files programmatically.
- Familiarize with scripting logic to streamline repetitive project maintenance work.

2. Tools

- **Python3** (Recommended: Python 3.10+)
- **Operating System:** Windows/Linux/Mac
- **Code Editor:** VS Code, PyCharm, or any preferred IDE
- **Terminal/Command Prompt**
- **Required Libraries:**
 - `os` (standard library)
 - `shutil` (for advanced file operations)
 - `pathlib` (optional)

3. Introduction

Python has become an essential tool in the digital IC design workflow due to its simple syntax, robust scripting capabilities, and extensive community support. It enables designers to automate repetitive tasks, manage large design files, and enhance productivity without the complexity of traditional scripting languages. In this lab, we focus on using Python for automating project file management tasks, with a special emphasis on the `os` library. The `os` library provides a powerful interface to interact with the operating system, allowing users to create and manage directories, access file metadata, rename or delete files, and traverse file systems. These capabilities are particularly valuable in digital IC design environments, where managing RTL code, testbenches, synthesis reports, and documentation files can become cumbersome. Through this lab, students will learn to use Python scripts to automatically generate project directory structures, organize synthesis runs, clean temporary files, and batch-generate module templates — all crucial tasks for maintaining an efficient and scalable design flow.



4. Lab Task

- Task 1: Setup and Installation
- Install Python:
 - [Windows/Linux/Mac] Download and install latest Python version from: <https://www.python.org/downloads/>
 - Confirm installation by typing **python --version** in cmd/terminal, it should output the installed version of the python.
 - Install VS Code (optional): <https://code.visualstudio.com/>
 - For guidance the following hyperlinked time slots are mentioned:
 - [00:03:22](#) Installing Python
 - [00:05:15](#) Python Interpreter
 - [00:07:12](#) Code Editors
 - Walkthrough this tutorial if you are a complete beginner to Python: <https://www.youtube.com/watch?v=yE9v9rt6ziw>
- Task 2: Create Project Directory Structure
 - Complete the following tutorials for os, shutil and pathlib libraries in python
 - Os library guide: <https://www.youtube.com/watch?v=tJxcKyFMTGo>
 - Shutil guide: <https://www.youtube.com/watch?v=sXzezlK0d7c>
 - Pathlib guide: <https://www.youtube.com/watch?v=yxa-DJuuTBI>
 - Using the above-mentioned libraries complete Task 3 and Task 4.
- Task 3: Create Project Directory Structure
 - Write a Python script to:
 - Create folders in your desired project directory: rtl/, tb/, scripts/, docs/, synthesis/, simulation/
 - Check if they already exist before creation
- Task 4: Auto-Generate Verilog Module Template
 - Create a script that:
 - Takes a module name as input, creates module_name.sv extension file inside rtl/ folder with a basic module template

Example:

```
module module_name(  
);  
  
endmodule
```

5. Submission Guidelines

Students must submit the following files in a compressed .rar archive:



- - Python scripts for each task (clearly named)
- - A short README.txt explaining script usage
- - Screenshots of terminal output for each task

6. References

- Python os module: <https://docs.python.org/3/library/os.html>
- Python shutil module: <https://docs.python.org/3/library/shutil.html>
- Python pathlib module: <https://docs.python.org/3/library/pathlib.html>
- Real Python: <https://realpython.com/working-with-files-in-python/>